

Customer Support - How it works

Home notes + DESIGN.md + source code | compact print

In one line - Customer raises issue -> assign to free agent with right skill -> agent resolves -> history saved.

Story - how this LLD works

Think of a fintech helpdesk. Issues have types (payment, gold, etc.). Agents have expertise. Assignment strategy picks the first free agent who knows that type. If nobody free, issue goes to waitlists. Resolve frees the agent and stores the issue in their history.

Who does what

Issue: The ticket (type, status, customer email, resolution).

Agent: Person with expertise set, optional current issue, waitlist, history.

IssueService: Create, filter, update, resolve issues.

AssignmentService: Asks strategy for an agent; else waitlist.

AssignmentStrategy: Default: first available + expertise match.

AgentService: Register agents and view work history.

Main pieces

Issue

- type, status
- assigned agent?

Assignment

- strategy picks
- or waitlist

Agent

- expertise
- busy?
- waitlist
- history

Resolve

- free agent
- save history

Step-by-step at runtime

1. createIssue(...) -> status OPEN, saved.
2. assignIssue(id) -> strategy scans agents.
3. Found free expert -> link agent <-> issue, mark work in progress style flow.
4. No free agent -> issue WAITING, pushed to waitlists of matching agents.
5. resolveIssue(id, text) -> RESOLVED, clear agent assignment, append to history.

Why this design (plain words)

Why Strategy? - Today first-match; tomorrow round-robin or least-loaded - swap class only.

Why separate services? - Creating tickets vs assigning vs agent admin stay readable.

Remember

- Same waiting issue may be added to ALL matching agents' waitlists (demo behavior).
- Waitlist is not auto-drained on resolve in this code - good improvement exercise.

OOP in this project

Encapsulation - Repos hide storage dictionaries.

Abstraction - assign(agents, issue) is the only assignment contract.

Inheritance - DefaultAssignmentStrategy implements AssignmentStrategy.

Polymorphism - AssignmentService calls strategy.assign blindly.

DESIGN.md - full file

Complete markdown from lld/customer_support/DESIGN.md (Q&A for learning).

Full DESIGN.md (same file as in this folder) - DESIGN.md

Designing a Customer Issue Resolution System - LLD Q&A

Run: `python -m lld.customer_support.main`

Patterns: Strategy · Repository · Service layer

Q1. What problem are we solving?

A. Design a fintech customer support system that:

Lets customers create issues (tickets) tied to transactions

Registers agents with domain expertise

Assigns issues to available matching agents

Supports issue lifecycle: OPEN -> IN_PROGRESS -> RESOLVED / WAITING

Filters issues and tracks agent work history

Waitlists issues when no free agent matches

Q2. What are the core entities?

- Entity - Responsibility
- Issue - Ticket id, type, status, resolution, assigned agent
- Agent - Expertise set, current assignment, waitlist (deque), history
- IssueService - Create, filter, update, resolve
- AgentService - Register agents, view history
- AssignmentService - Delegates to assignment strategy; waitlist fallback

Q3. How is the package structured?

```
customer_support/
|-- model/      # Issue, Agent
|-- repository/ # IssueRepository, AgentRepository
|-- service/    # IssueService, AgentService, AssignmentService
|-- strategy/   # AssignmentStrategy + DefaultAssignmentStrategy
|-- enums/     # IssueStatus, IssueType
```

Q4. Why Strategy for assignment?

A. How we pick an agent can change without touching AssignmentService.

```
class AssignmentStrategy(ABC):
    def assign(self, agents, issue) -> Optional[Agent]: ...
```

Default: first available agent whose expertise contains the issue type.

Easy to add: RoundRobin, LeastLoaded, SkillWeighted.

Q5. What are the main flows?

Create issue

```
IssueService.create_issue(...) -> Issue(OPEN) -> save
```

Assign

```
AssignmentService.assign_issue(id)
-> strategy.assign(agents, issue)
-> found: link agent ↔ issue
-> else: add to waitlist of matching agents, status WAITING
```

Resolve

```
IssueService.resolve_issue(id, resolution)
-> RESOLVED + resolution
-> append to agent history, free agent
```

Q6. Why Repository?

A. Services talk to IssueRepository / AgentRepository (in-memory dicts). Swap for DB/Redis later without changing business logic.

Q7. How do you extend this design?

- Add... - How
- New assignment algo - Implement AssignmentStrategy
- New issue type - Add to IssueType enum
- Auto-drain waitlist - On resolve, pop next waitlisted issue for that agent
- Notifications - Observer on status changes

Q8. Common interview follow-ups

Q. Waitlist behavior?

Same issue can be added to all matching agents' waitlists - not a single shared queue. Discuss draining on free-agent as a follow-up.

Q. How does filtering work?

`get_issues(filters)` - e.g. by email or type; type string normalizes spaces to underscores for enum match.

Q. Double assignment?

Guarded by `agent.is_available()` only - no distributed lock in this demo.

Source code - Ild/customer_support/

12 Python files below (compact font to save pages). Read with the notes: find the class name from the cast, then open that file here.

- Tip: start at main.py, then service/, then model/ + strategy/.

FILE: main.py (79 lines)

```

1| from .enums.issue_status import IssueStatus
2| from .enums.issue_type import IssueType
3| from .repository.agent_repository import AgentRepository
4| from .repository.issue_repository import IssueRepository
5| from .service.agent_service import AgentService
6| from .service.assignment_service import AssignmentService
7| from .service.issue_service import IssueService
8| from .strategy.assignment.default_assignment_strategy import DefaultAssignmentStrategy
9|
10|
11| def main() -> None:
12|     agent_repository = AgentRepository()
13|     issue_repository = IssueRepository()
14|
15|     agent_service = AgentService(agent_repository)
16|     issue_service = IssueService(issue_repository, agent_repository)
17|     assignment_service = AssignmentService(
18|         agent_repository, issue_repository, DefaultAssignmentStrategy()
19|     )
20|
21|     i1 = issue_service.create_issue(
22|         "T1",
23|         IssueType.PAYMENT_RELATED,
24|         "Payment Failed",
25|         "My payment failed but money is debited",
26|         "testUser1@test.com",
27|     )
28|     i2 = issue_service.create_issue(
29|         "T2",
30|         IssueType.MUTUAL_FUND_RELATED,
31|         "Purchase Failed",
32|         "Unable to purchase Mutual Fund",
33|         "testUser2@test.com",
34|     )
35|     i3 = issue_service.create_issue(
36|         "T3",
37|         IssueType.PAYMENT_RELATED,
38|         "Payment Failed",
39|         "My payment failed but money is debited",
40|         "testUser2@test.com",
41|     )
42|
43|     agent_service.add_agent(
44|         "agent1@test.com",
45|         "Agent 1",
46|         [IssueType.PAYMENT_RELATED, IssueType.GOLD_RELATED],
47|     )
48|     agent_service.add_agent(
49|         "agent2@test.com",
50|         "Agent 2",
51|         [IssueType.PAYMENT_RELATED],
52|     )
53|
54|     assignment_service.assign_issue(i1.id)
55|     assignment_service.assign_issue(i2.id)
56|     assignment_service.assign_issue(i3.id)
57|
58|     print("\n--- Issues for testUser2@test.com ---")
59|     for issue in issue_service.get_issues({"email": "testUser2@test.com"}):
60|         print(issue)
61|
62|     print("\n--- Payment Related Issues ---")
63|     for issue in issue_service.get_issues({"type": "Payment Related"}):
64|         print(issue)
65|
66|     issue_service.update_issue(
67|         i3.id, IssueStatus.IN_PROGRESS, "Waiting for payment confirmation"
68|     )
69|
70|     issue_service.resolve_issue(
71|         i3.id, "Payment failed. Debited amount will be reversed."
72|     )
73|
74|     print("\n--- Agent Work History ---")
75|     agent_service.view_agents_work_history()
76|
77|
78| if __name__ == "__main__":
79|     main()

```

FILE: service/agent_service.py (21 lines)

(cont.) service/agent_service.py

```

1| import uuid
2| from typing import List
3|
4| from ..enums.issue_type import IssueType
5| from ..model.agent import Agent
6| from ..repository.agent_repository import AgentRepository
7|
8|
9| class AgentService:
10|     def __init__(self, agent_repository: AgentRepository) -> None:
11|         self._agent_repository = agent_repository
12|
13|     def add_agent(self, email: str, name: str, issue_types: List[IssueType]) -> None:
14|         agent_id = "A" + str(uuid.uuid4())[6]
15|         agent = Agent(id=agent_id, email=email, name=name, expertise=set(issue_types))
16|         self._agent_repository.save(agent)
17|         print(f">>> Agent {agent_id} created")
18|
19|     def view_agents_work_history(self) -> None:
20|         for agent in self._agent_repository.get_all():
21|             print(f">>> {agent.id} -> {agent.history}")

```

FILE: service/assignment_service.py (40 lines)

```

1| from typing import List
2|
3| from ..enums.issue_status import IssueStatus
4| from ..model.agent import Agent
5| from ..repository.agent_repository import AgentRepository
6| from ..repository.issue_repository import IssueRepository
7| from ..strategy.assignment.assignment_strategy import AssignmentStrategy
8|
9|
10| class AssignmentService:
11|     def __init__(
12|         self,
13|         agent_repository: AgentRepository,
14|         issue_repository: IssueRepository,
15|         strategy: AssignmentStrategy,
16|     ) -> None:
17|         self._agent_repository = agent_repository
18|         self._issue_repository = issue_repository
19|         self._strategy = strategy
20|
21|     def assign_issue(self, issue_id: str) -> None:
22|         issue = self._issue_repository.get_by_id(issue_id)
23|         if issue is None:
24|             raise ValueError("Issue not found")
25|
26|         agents: List[Agent] = list(self._agent_repository.get_all())
27|         assigned = self._strategy.assign(agents, issue)
28|
29|         if assigned is not None:
30|             assigned.assigned_issue_id = issue.id
31|             issue.assigned_agent_id = assigned.id
32|             print(f">>> Issue {issue_id} assigned to agent {assigned.id}")
33|         else:
34|             for agent in agents:
35|                 if issue.issue_type in agent.expertise:
36|                     agent.wait_list.append(issue.id)
37|                     issue.status = IssueStatus.WAITING
38|                     print(f">>> Issue {issue_id} added to waitlist of Agent {agent.id}")
39|             return
40|         print(f">>> No agent found with expertise for issue {issue_id}")

```

FILE: service/issue_service.py (82 lines)

```

1| from typing import Dict, List
2|
3| from ..enums.issue_status import IssueStatus
4| from ..enums.issue_type import IssueType
5| from ..model.issue import Issue
6| from ..repository.agent_repository import AgentRepository
7| from ..repository.issue_repository import IssueRepository
8|
9|
10| class IssueService:
11|     def __init__(
12|         self,
13|         issue_repository: IssueRepository,
14|         agent_repository: AgentRepository,
15|     ) -> None:
16|         self._issue_repository = issue_repository
17|         self._agent_repository = agent_repository
18|
19|     def create_issue(
20|         self,
21|         transaction_id: str,
22|         issue_type: IssueType,
23|         subject: str,
24|         description: str,
25|         email: str,
26|     ) -> Issue:
27|         issue = Issue(
28|             transaction_id=transaction_id,
29|             issue_type=issue_type,
30|             subject=subject,
31|             description=description,
32|             email=email,
33|         )

```

```

34|     self._issue_repository.save(issue)
35|     print(
36|         f">>> Issue {issue.id} created against transaction \"{transaction_id}\"
37|     )
38|     return issue
39|
40| # {"email": "testUser2@test.com"}
41| # {"type": "Payment Related"}
42| # {"status": "open"}
43| def get_issues(self, filter: Dict[str, str]) -> List[Issue]:
44|     result = []
45|     for issue in self._issue_repository.get_all():
46|         if "email" in filter and issue.email.lower() != filter["email"].lower():
47|             continue
48|         if "type" in filter:
49|             type_filter = filter["type"].replace(" ", "_")
50|             if issue.issue_type.name.lower() != type_filter.lower():
51|                 continue
52|         if "status" in filter:
53|             if issue.status.name.lower() != filter["status"].lower():
54|                 continue
55|         result.append(issue)
56|     return result
57|
58| def update_issue(
59|     self, issue_id: str, status: IssueStatus, resolution: str
60| ) -> None:
61|     issue = self._issue_repository.get_by_id(issue_id)
62|     if issue is None:
63|         raise ValueError("Issue not found")
64|     issue.status = status
65|     issue.resolution = resolution
66|     print(f">>> {issue_id} status updated to {issue.status.name}")
67|
68| def resolve_issue(self, issue_id: str, resolution: str) -> None:
69|     issue = self._issue_repository.get_by_id(issue_id)
70|     if issue is None:
71|         raise ValueError("Issue not found")
72|
73|     issue.status = IssueStatus.RESOLVED
74|     issue.resolution = resolution
75|
76|     if issue.assigned_agent_id is not None:
77|         agent = self._agent_repository.get_by_id(issue.assigned_agent_id)
78|         if agent is not None:
79|             agent.history.append(issue.id)
80|             agent.assigned_issue_id = None
81|
82|     print(f">>> {issue_id} issue marked resolved")

```

FILE: model/agent.py (19 lines)

```

1| from collections import deque
2| from dataclasses import dataclass, field
3| from typing import Optional, Set
4|
5| from ..enums.issue_type import IssueType
6|
7|
8| @dataclass
9| class Agent:
10|     id: str
11|     email: str
12|     name: str
13|     expertise: Set[IssueType]
14|     assigned_issue_id: Optional[str] = None
15|     wait_list: deque = field(default_factory=deque)
16|     history: list = field(default_factory=list)
17|
18|     def is_available(self) -> bool:
19|         return self.assigned_issue_id is None

```

FILE: model/issue.py (29 lines)

```

1| import uuid
2| from dataclasses import dataclass, field
3| from typing import Optional
4|
5| from ..enums.issue_status import IssueStatus
6| from ..enums.issue_type import IssueType
7|
8|
9| @dataclass
10| class Issue:
11|     transaction_id: str
12|     issue_type: IssueType
13|     subject: str
14|     description: str
15|     email: str
16|     id: str = field(init=False)
17|     status: IssueStatus = field(init=False, default=IssueStatus.OPEN)
18|     resolution: Optional[str] = None
19|     assigned_agent_id: Optional[str] = None
20|
21|     def __post_init__(self) -> None:
22|         self.id = "I" + str(uuid.uuid4())[6]
23|
24|     def __str__(self) -> str:
25|         return (
26|             f'{self.id} {{"{self.transaction_id}", "{self.issue_type.name}", '
27|             f'"{self.subject}", "{self.description}", "{self.email}", '

```

```
28|         f'{self.status.name}')}
29|     )
```

FILE: strategy/assignment/assignment_strategy.py (11 lines)

```
1| from abc import ABC, abstractmethod
2| from typing import List, Optional
3|
4| from ...model.agent import Agent
5| from ...model.issue import Issue
6|
7|
8| class AssignmentStrategy(ABC):
9|     @abstractmethod
10|     def assign(self, agents: List[Agent], issue: Issue) -> Optional[Agent]:
11|         pass
```

FILE: strategy/assignment/default_assignment_strategy.py (13 lines)

```
1| from typing import List, Optional
2|
3| from ...model.agent import Agent
4| from ...model.issue import Issue
5| from .assignment_strategy import AssignmentStrategy
6|
7|
8| class DefaultAssignmentStrategy(AssignmentStrategy):
9|     def assign(self, agents: List[Agent], issue: Issue) -> Optional[Agent]:
10|         for agent in agents:
11|             if agent.is_available() and issue.issue_type in agent.expertise:
12|                 return agent
13|         return None
```

FILE: repository/agent_repository.py (17 lines)

```
1| from typing import Collection, Dict, Optional
2|
3| from ..model.agent import Agent
4|
5|
6| class AgentRepository:
7|     def __init__(self) -> None:
8|         self._agents: Dict[str, Agent] = {}
9|
10|     def save(self, agent: Agent) -> None:
11|         self._agents[agent.id] = agent
12|
13|     def get_by_id(self, id: str) -> Optional[Agent]:
14|         return self._agents.get(id)
15|
16|     def get_all(self) -> Collection[Agent]:
17|         return self._agents.values()
```

FILE: repository/issue_repository.py (17 lines)

```
1| from typing import Collection, Dict, Optional
2|
3| from ..model.issue import Issue
4|
5|
6| class IssueRepository:
7|     def __init__(self) -> None:
8|         self._issues: Dict[str, Issue] = {}
9|
10|     def save(self, issue: Issue) -> None:
11|         self._issues[issue.id] = issue
12|
13|     def get_by_id(self, id: str) -> Optional[Issue]:
14|         return self._issues.get(id)
15|
16|     def get_all(self) -> Collection[Issue]:
17|         return self._issues.values()
```

FILE: enums/issue_status.py (8 lines)

```
1| from enum import Enum
2|
3|
4| class IssueStatus(Enum):
5|     OPEN = "OPEN"
6|     IN_PROGRESS = "IN_PROGRESS"
7|     RESOLVED = "RESOLVED"
8|     WAITING = "WAITING"
```

FILE: enums/issue_type.py (8 lines)

```
1| from enum import Enum
2|
3|
4| class IssueType(Enum):
5|     PAYMENT_RELATED = "PAYMENT_RELATED"
6|     MUTUAL_FUND_RELATED = "MUTUAL_FUND_RELATED"
7|     GOLD_RELATED = "GOLD_RELATED"
8|     INSURANCE_RELATED = "INSURANCE_RELATED"
```